

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8

дисциплина: Архитектура компьютера

Студент: Томилова Валентина

Группа: НКАбд-06-25

МОСКВА

2025 г.

Оглавление

<u>1 Цель работы</u>	3
<u>2 Задания</u>	4
<u>3 Теоретическое введение</u>	5
<u>4 Выполнение работы</u>	6
<u>Реализация циклов в NASM</u>	6
<u>Обработка аргументов командной строки</u>	10
<u>Задание для самостоятельной работы</u>	13
<u>5 Выводы</u>	16
<u>Список литературы</u>	17

Список иллюстраций

Рисунок 0.1 создание каталога	7
Рисунок 0.2 переход в каталог.....	7
Рисунок 0.3 создание файла lab8-1.asm.....	7
Рисунок 0.4 запись программы.....	8
Рисунок 0.5 создание исполняемого файла	8
Рисунок 0.6 создание исполняемого файла	8
Рисунок 0.7 запуск исполняемого файла	8
Рисунок 0.8 проверка работы программы.....	9
Рисунок 0.9 изменение программы	9
Рисунок 0.10 создание исполняемого файла	10
Рисунок 0.11 запуск исполняемого файла	10
Рисунок 0.12 проверка работы программы.....	10
Рисунок 0.13 изменение программы	11
Рисунок 0.14 создание исполняемого файла	11
Рисунок 0.15 создание исполняемого файла	11
Рисунок 0.16 проверка работы программы.....	12
Рисунок 0.17 запись программы.....	12
Рисунок 0.18 создание исполняемого файла	13
Рисунок 0.19 проверка работы программы.....	13
Рисунок 0.20 создание файла	13
Рисунок 0.21 запись программы.....	14
Рисунок 0.22 создание исполняемого файла	14
Рисунок 0.23 создание исполняемого файла	14
Рисунок 0.24 запуск исполняемого кода.....	14
Рисунок 0.25 проверка работы программы.....	15
Рисунок 0.26 изменение программы	15
Рисунок 0.27 создание исполняемого файла	15
Рисунок 0.28 создание исполняемого файла	15
Рисунок 0.29 проверка работы программы.....	16
Рисунок 0.30 запись кода	16
Рисунок 0.31 создание исполняемого файла	17
Рисунок 0.32 создание исполняемого файла	17
Рисунок 0.33 запуск исполняемого файла	17
Рисунок 0.34 проверка работы программы.....	17

1 Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

2 Задания

1. Реализация циклом в NASM
2. Обработка аргументов командной строки
3. Самостоятельное написание программы по материалам лабораторной работы

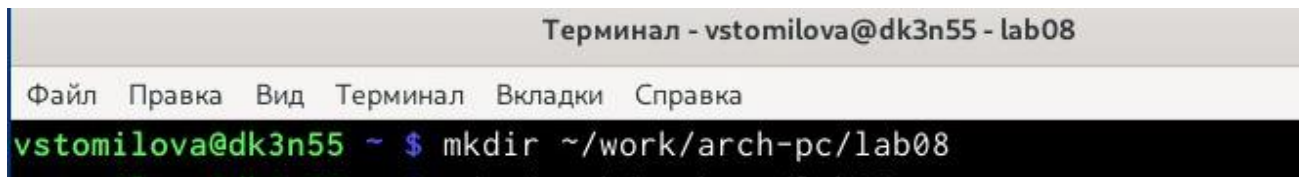
3 Теоретическое введение

Стек — это структура данных, организованная по принципу LIFO («Last In — First Out» или «последним пришёл—первым ушёл»). Стек является частью архитектуры процессора и реализована аппаратном уровне. Для работы со стеком в процессоре есть специальные регистры (ss, bp, sp) и команды. Основной функцией стека является функция сохранения адресов возврата и передачи аргументов при вызове процедур. Кроме того, в нём выделяется память для локальных переменных и могут временно храниться значения регистров

4 Выполнение работы

Реализация циклов в NASM

Создадим каталог для программ лабораторной работы №8, перейдем в него и создадим файл lab8-1.asm (рисунок 0.1-0.3)



Терминал - vstomilova@dk3n55 - lab08

Файл Правка Вид Терминал Вкладки Справка

```
vstomilova@dk3n55 ~ $ mkdir ~/work/arch-pc/lab08
```

Рисунок 0.1 создание каталога

```
vstomilova@dk3n55 ~ $ cd ~/work/arch-pc/lab08
```

Рисунок 0.2 переход в каталог

```
vstomilova@dk3n55 ~ $ mkdir ~/work/arch-pc/lab08
vstomilova@dk3n55 ~ $ cd ~/work/arch-pc/lab08
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ touch lab8-1.asm
vstomilova@dk3n55 ~/work/arch-pc/lab08 $
```

Рисунок 0.3 создание файла lab8-1.asm

Введем в файл lab8-1.asm текст программы из листинга(рисунок 0.4)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ gedit lab8-1.asm

Открыть  ▼  [+]  lab8-1.asm  ~/work/arch-pc/lab08  Сохранить  ≡

1 %include 'in_out.asm'
2 SECTION .data
3 msg1 db 'Введите N: ',0h
4
5
6 SECTION .bss
7 N: resb 10
8 SECTION .text
9 global _start
10 _start:
11 ; ----- Вывод сообщения 'Введите N: '
12 mov eax,msg1
13 call sprint
14 ; ----- Ввод 'N'
15 mov ecx, N
16 mov edx, 10
17 call sread
18 ; ----- Преобразование 'N' из символа в число
19 mov eax,N
20 call atoi
21 mov [N],eax
22 ; ----- Организация цикла
23 mov ecx,[N] ; Счетчик цикла, 'ecx=N'
24 label:
25 mov [N],ecx
26 mov eax,[N]
27 call iprintLF ; Вывод значения 'N'
28 loop label ; 'ecx=ecx-1' и если 'ecx' не '0'
29 ; переход на 'label'
30 call quit
```

Рисунок 0.4 запись программы

Создадим исполняемый файл и проверим его работу (рисунок 0.5-0.8)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-1.asm
```

Рисунок 0.5 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-1.asm
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-1 lab8-1.o
```

Рисунок 0.6 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-1
Введите N: 10
```

Рисунок 0.7 запуск исполняемого файла

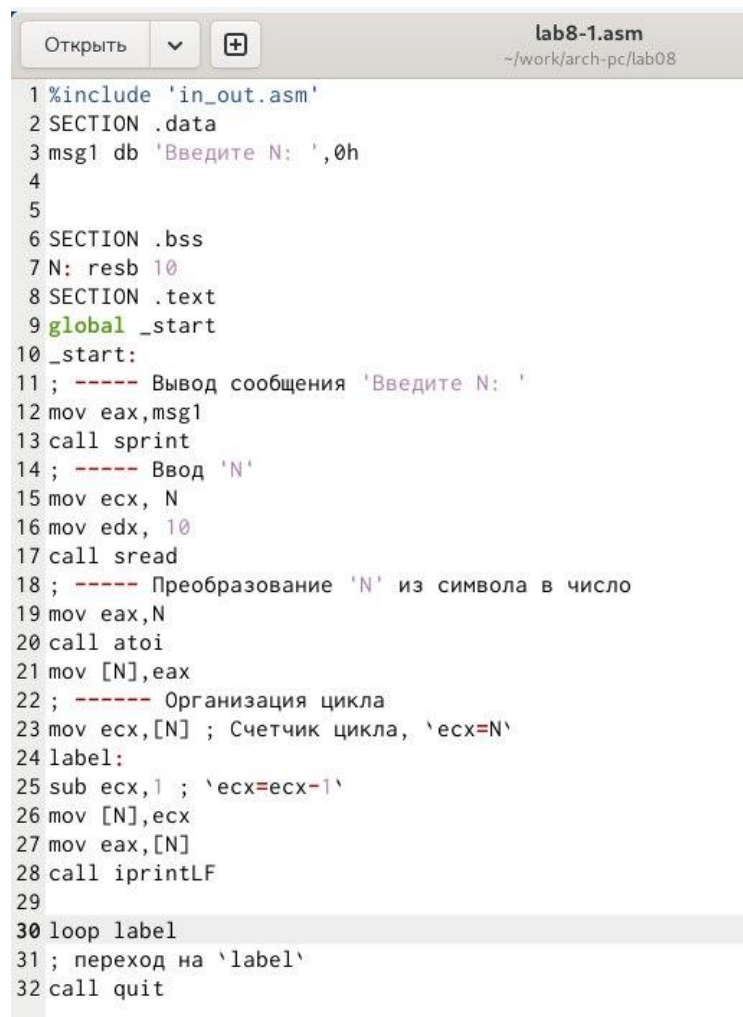
```

vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-1
Введите N: 10
10
9
8
7
6
5
4
3
2
1
vstomilova@dk3n55 ~/work/arch-pc/lab08 $

```

Рисунок 0.8 проверка работы программы

Изменим текст программы добавив изменение значения регистра `ecx` в цикле (рисунок 0.9-0.12)



```

lab8-1.asm
~/work/arch-pc/lab08

1 %include 'in_out.asm'
2 SECTION .data
3 msg1 db 'Введите N: ',0h
4
5
6 SECTION .bss
7 N: resb 10
8 SECTION .text
9 global _start
10 _start:
11 ; ----- Вывод сообщения 'Введите N: '
12 mov eax,msg1
13 call sprint
14 ; ----- Ввод 'N'
15 mov ecx, N
16 mov edx, 10
17 call sread
18 ; ----- Преобразование 'N' из символа в число
19 mov eax,N
20 call atoi
21 mov [N],eax
22 ; ----- Организация цикла
23 mov ecx,[N] ; Счетчик цикла, 'ecx=N'
24 label:
25 sub ecx,1 ; 'ecx=ecx-1'
26 mov [N],ecx
27 mov eax,[N]
28 call iprintLF
29
30 loop label
31 ; переход на 'label'
32 call quit

```

Рисунок 0.9 изменение программы

Создадим исполняемый файл и проверьте его работу

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ gedit lab8-1.asm  
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-1.asm
```

Рисунок 0.10 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-1 lab8-1.o
```

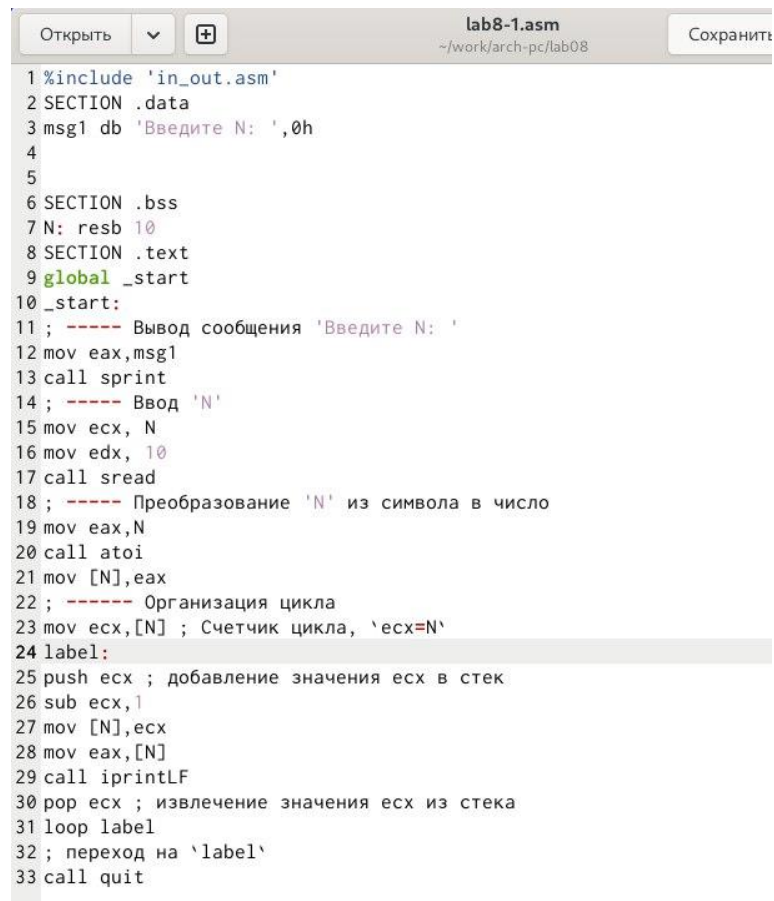
Рисунок 0.11 запуск исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-1  
Введите N: 10  
9  
7  
5  
3  
1  
vstomilova@dk3n55 ~/work/arch-pc/lab08 $
```

Рисунок 0.12 проверка работы программы

Количество итераций уменьшается вдвое ввиду того, что регистр `ecx` на каждой итерации стал меньше на 2 значения

Внесем изменения в текст программы, добавив команды `push` и `pop` для сохранения значения счетчика цикла `loop` (рисунок 0.13)



```
1 %include 'in_out.asm'
2 SECTION .data
3 msg1 db 'Введите N: ',0h
4
5
6 SECTION .bss
7 N: resb 10
8 SECTION .text
9 global _start
10 _start:
11 ; ----- Вывод сообщения 'Введите N: '
12 mov eax,msg1
13 call sprint
14 ; ----- Ввод 'N'
15 mov ecx, N
16 mov edx, 10
17 call sread
18 ; ----- Преобразование 'N' из символа в число
19 mov eax,N
20 call atoi
21 mov [N],eax
22 ; ----- Организация цикла
23 mov ecx,[N] ; Счетчик цикла, 'ecx=N'
24 label:
25 push ecx ; добавление значения ecx в стек
26 sub ecx,1
27 mov [N],ecx
28 mov eax,[N]
29 call iprintLF
30 pop ecx ; извлечение значения ecx из стека
31 loop label
32 ; переход на 'label'
33 call quit
```

Рисунок 0.13 изменение программы

Создадим исполняемый файл и проверим его работу (рисунок 0.14-0.16)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ gedit lab8-1.asm
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-1.asm
```

Рисунок 0.14 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-1 lab8-1.o
```

Рисунок 0.15 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-1
Введите N: 10
9
8
7
6
5
4
3
2
1
0
vstomilova@dk3n55 ~/work/arch-pc/lab08 $
```

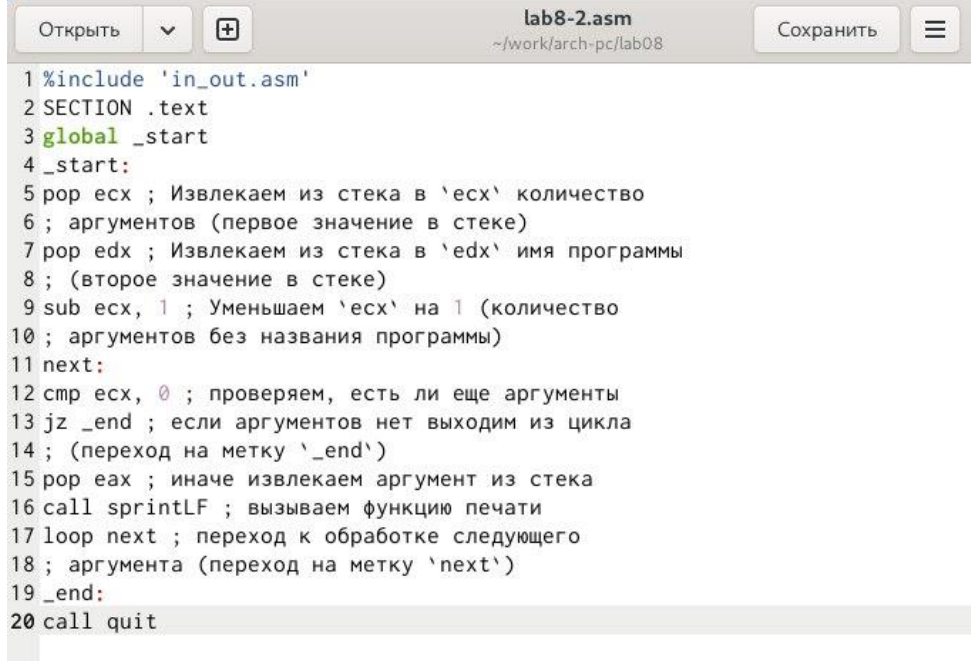
Рисунок 0.16 проверка работы программы

Произошло смещение выводимых чисел на -1, но теперь количество итераций совпадает введенному N

Обработка аргументов командной строки

Создадим файл lab8-2.asm в каталоге ~/work/arch-pc/lab08 и введем в него текст программы из листинга (рисунок 0.17)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ touch lab8-2.asm
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ gedit lab8-2.asm
```



```
1 %include 'in_out.asm'
2 SECTION .text
3 global _start
4 _start:
5 pop ecx ; Извлекаем из стека в 'ecx' количество
6 ; аргументов (первое значение в стеке)
7 pop edx ; Извлекаем из стека в 'edx' имя программы
8 ; (второе значение в стеке)
9 sub ecx, 1 ; Уменьшаем 'ecx' на 1 (количество
10 ; аргументов без названия программы)
11 next:
12 cmp ecx, 0 ; проверяем, есть ли еще аргументы
13 jz _end ; если аргументов нет выходим из цикла
14 ; (переход на метку '_end')
15 pop eax ; иначе извлекаем аргумент из стека
16 call sprintf ; вызываем функцию печати
17 loop next ; переход к обработке следующего
18 ; аргумента (переход на метку 'next')
19 _end:
20 call quit
```

Рисунок 0.17 запись программы

Создадим исполняемый файл и запустим его, указав аргументы (рисунок

0.18-0.19)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-2.asm  
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-2 lab8-2.o
```

Рисунок 0.18 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-2 аргумент1 аргумент 2 'аргумент 3'  
аргумент1  
аргумент  
2  
аргумент 3  
vstomilova@dk3n55 ~/work/arch-pc/lab08 $
```

Рисунок 0.19 проверка работы программы

Программа обработала то же количество аргументов, что и было введено

Создадим файл lab8-3.asm в каталоге ~/work/arch pc/lab08 и введем в него текст программы из листинга (рисунок 0.20-0.21)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ touch lab8-3.asm  
vstomilova@dk3n55 ~/work/arch-pc/lab08 $
```

Рисунок 0.20 создание файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ gedit lab8-3.asm

Открыть  lab8-3.asm  Сохранить
~/work/arch-pc/lab08

1 %include 'in_out.asm'
2 SECTION .data
3 msg db "Результат: ",0
4 SECTION .text
5 global _start
6 _start:
7
8 pop ecx ; Извлекаем из стека в 'ecx' количество
9 ; аргументов (первое значение в стеке)
10 pop edx ; Извлекаем из стека в 'edx' имя программы
11 ; (второе значение в стеке)
12 sub ecx,1 ; Уменьшаем 'ecx' на 1 (количество
13 ; аргументов без названия программы)
14 mov esi, 0 ; Используем 'esi' для хранения
15 ; промежуточных сумм
16 next:
17 cmp ecx,0h ; проверяем, есть ли еще аргументы
18 jz _end ; если аргументов нет выходим из цикла
19 ; (переход на метку '_end')
20 pop eax ; иначе извлекаем следующий аргумент из стека
21 call atoi ; преобразуем символ в число
22 add esi,eax ; добавляем к промежуточной сумме
23 ; след. аргумент 'esi=esi+eax'
24 loop next ; переход к обработке следующего аргумента
25 _end:
26 mov eax, msg ; вывод сообщения "Результат: "
27 call sprint
28 mov eax, esi ; записываем сумму в регистр 'eax'
29 call iprintLF ; печать результата
30 call quit
```

Рисунок 0.21 запись программы

Создадим исполняемый файл и запустим его, указав аргументы (рисунок 0.22-0.25)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-3.asm
```

Рисунок 0.22 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-3.asm
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-3 lab8-3.o
```

Рисунок 0.23 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-3 12 13 7 10 5
```

Рисунок 0.24 запуск исполняемого кода

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-3 12 13 7 10 5
Результат: 47
vstomilova@dk3n55 ~/work/arch-pc/lab08 $
```

Рисунок 0.25 проверка работы программы

Изменим текст программы из листинга для вычисления произведения аргументов командной строки (рисунок 0.26-0.29)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ gedit lab8-3.asm

*lab8-3.asm
~/work/arch-pc/lab08

1 %include 'in_out.asm'
2 SECTION .data
3 msg db "Результат: ",0
4 SECTION .text
5 GLOBAL _start
6 _start:
7 pop ecx
8 pop edx
9 sub ecx,1
10 mov esi,1
11
12 next:
13 cmp ecx,0h
14 jz _end
15 pop eax
16 call atoi
17 mul esi
18 mov esi, eax
19
20 loop next
21
22 _end:
23 mov eax, msg
24 call sprint
25 mov eax, esi
26 call iprintLF
27 call quit
```

Рисунок 0.26 изменение программы

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-3.asm
```

Рисунок 0.27 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-3 lab8-3.o
```

Рисунок 0.28 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-3 12 13 7 10 5
Результат: 54600
vstomilova@dk3n55 ~/work/arch-pc/lab08 $
```

Рисунок 0.29 проверка работы программы

Задание для самостоятельной работы

Напишем программу, которая находит сумму значений функции $f(x)$ для $x = 1, 2, \dots, n$, т.е. программа должна выводить значение $f(1) + f(2) + \dots + f(n)$. Выражение для $f(x) = 3(10+x)$. (рисунок 0.30)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ gedit lab8-4.asm
*lab8-4.asm
~/work/arch-pc/lab08
Сохранить

1 %include 'in_out.asm'
2 SECTION .data
3 msg_func db "Функция:f(x) = 3(10+x)", 0
4 msg_result db "Результат: ", 0
5 SECTION .text
6 GLOBAL _start
7
8 _start:
9 mov eax, msg_func
10 call sprintf
11 pop ecx
12 pop edx
13 sub ecx, 1
14 mov esi, 0
15
16 next:
17 cmp ecx, 0h
18 jz _end
19 pop eax
20 call atoi
21
22 add eax, 10
23 mov ebx, 3
24 mul ebx
25
26 add esi, eax
27 loop next
28
29 _end:
30 mov eax, msg_result
31 call sprintf
32 mov eax, esi
33 call iprintLF
34 call quit
```

Рисунок 0.30 запись кода

Создадим исполняемый файл и проверьте его работу на нескольких наборах $n = 1, 2, \dots, 4$ (рисунок 0.31-0.34)

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-4.asm
```

Рисунок 0.31 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-4 lab8-4.o  
vstomilova@dk3n55 ~/work/arch-pc/lab08 $
```

Рисунок 0.32 создание исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-4 1 2 3 4
```

Рисунок 0.33 запуск исполняемого файла

```
vstomilova@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-4 1 2 3 4  
Функция:  $f(x) = 3(10+x)$   
Результат: 150  
vstomilova@dk3n55 ~/work/arch-pc/lab08 $
```

Рисунок 0.34 проверка работы программы

5 Выводы

Я приобрела навыки написания программ с использованием циклов и обработкой аргументов командной строки.

Список литературы

1. GDB:TheGNUProjectDebugger.—URL:<https://www.gnu.org/software/gdb/>.
2. GNUBashManual.—2016.—URL:<https://www.gnu.org/software/bash/manual/>.
3. Midnight CommanderDevelopment Center.—2021.—URL: <https://midnight-commander.org/>.
4. NASMAssemblyLanguageTutorials.—2021.—URL: <https://asmtutor.com/>.
5. Newham C. Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 с. —(In a Nutshell). —ISBN 0596009658.—URL: [http://www.amazon.com/Learning bash-Shell-Programming-Nutshell/dp/0596009658](http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658).
6. RobbinsA. Bash Pocket Reference.—O'Reilly Media,2016.—156 с.—ISBN 978-1491941591.
7. TheNASMdocumentation.—2021.—URL:<https://www.nasm.us/docs.php>.
8. Zarrelli G. Mastering Bash.—Packt Publishing,2017.—502 с.—ISBN 9781784396879.
9. Колдаев В. Д.,Лупин С. А.Архитектура ЭВМ.—М. : Форум,2018.
10. Куляс О. Л.,Никитин К. А. Курс программирования наASSEMBLER.— М. : Солон-Пресс, 2017.
11. НовожиловО.П.Архитектура ЭВМисистем.—М.:Юрайт,2016.
12. Расширенныйассемблер:NASM.—2021.—URL:<https://www.opennet.ru/docs/RUS/nasm/>.
13. Робачевский А.,Немнюгин С.,Стесик О. Операционная система UNIX.— 2-е изд.—БХВ Петербург, 2010.—656 с.—ISBN 978-5-94157-538-1.
14. СтоляровА.Программированиеязыкеассемблера NASMдляОСUnix.—2-еизд.— М. : МАКСПресс,2011.—URL:http://www.stolyarov.info/books/asm_unix.
15. Таненбаум Э. Архитектура компьютера. — 6-е изд. — СПб. : Питер, 2013. — 874 с. — (Классика Computer Science).
16. Таненбаум Э.,Бос Х. Современные операционные системы.—4-е изд.— СПб.: Питер, 2015. —1120 с.—(Классика Computer Science)